

Compiler-Chosen Recomputation (CCR)

MILP-Based Activation Memory Management with Unified Save, Compress, and Recompute Decisions in NeuralScript

Authors: Brandon Wiemer **Date:** April 2026 **Status:** Research Proposal

Abstract

We propose **CCR (Compiler-Chosen Recomputation)**, a first-class activation memory management technique for NSL that unifies three memory strategies — save-in-place, save-with-compression, and recompute-on-demand — into a single compile-time decision framework. Unlike existing approaches that apply rule-based heuristics (Megatron's full/selective recomputation) or one-dimensional automatic solvers (PyTorch's `activation_memory_budget` API), CCR treats activation management as a joint memory-compute MILP over the entire model graph, parameterized by tensor size, recompute cost, downstream usage count, outlier distribution, and CFIP interior-format eligibility. The MILP is solved once at compile time using the repetitive-block structure of transformer models to keep the search space tractable. CCR composes with CSHA's tiered attention (Tier A/B/C become points in CCR's decision space), CFIP's interior formats (CCR can save an activation in a compressed CFIP format rather than full precision), FASE's gradient-buffer elimination (freed memory feeds CCR's save budget), CPFT's frozen layers (frozen activation decisions are pruned from the MILP), and CTQS's phase-transition system (phase boundaries trigger MILP re-solve). Expected impact: 2-5× activation memory reduction at 2-10% compute overhead on typical training workloads, with monotonic memory-vs-speed tradeoff controllable by a single `@checkpoint(budget=...)` annotation.

1. The Activation Memory Wall

1.1 Where activation memory goes

In modern LLM training, activation memory dominates GPU memory at typical batch sizes:

Component	7B model, batch=4, seq=2048	Share
Model parameters (FP16)	14 GB	11%
Gradient buffer (FP16)	14 GB	11%
Optimizer state (FP32 Adam)	56 GB	44%
Activations (FP16)	42 GB	33%
Total	126 GB	

Activations grow quadratically with sequence length (through attention intermediates) and linearly with batch size. At seq=8192, batch=4, activations balloon to approximately 180 GB — exceeding H100 capacity before any parallelism is applied. The Adacc paper (2025) reports that activations occupy 66% of GPU memory for GPT training on V100 at batch=4, and 76% at batch=8.

NSL's existing techniques address three of the four components:

- **FASE Deferred** eliminates the gradient buffer (~11% saved).
- **CFTP / CFIP / CTQS** reduce parameter memory (~11% saved, varies with aggressiveness).
- **CPDT ZeRO-style sharding** splits optimizer state across ranks (~44% / world_size).

Activations remain unaddressed. CSHA Tier C's save-activations reduces attention activations but only for attention; it does not address MLP, LayerNorm, embedding, or residual activations.

1.2 The fundamental tradeoff

For any activation tensor, there are three options:

1. **Save** in full precision (cheapest compute, highest memory).
2. **Save with compression** (moderate compute overhead for quantize/dequant, reduced memory).
3. **Recompute** during backward (highest compute, lowest memory — just the inputs).

The optimal choice depends on per-tensor properties:

- **Tensor size:** larger tensors benefit more from recomputation or compression.
- **Recompute cost:** measured in FLOPs needed to reproduce the tensor from its inputs. Cheap to recompute (softmax, LayerNorm) → recompute is free memory. Expensive to recompute (large MLP matmul) → save preferred.
- **Downstream use count:** tensors consumed by many backward paths get recomputed multiple times; higher use count pushes toward save.
- **Quantization sensitivity:** tensors with heavy outliers (activations with attention sink tokens) lose accuracy when naively compressed; compression-with-outlier-handling or save-uncompressed preferred.
- **CFIP format eligibility:** kernel-interior tensors are already eligible for CFIP formats; boundary tensors need explicit compression.

Existing frameworks make these decisions by rules (Megatron: "checkpoint the transformer layer input") or by runtime heuristics (PyTorch's memory budget). Neither exploits the repeated-block structure of transformers, and neither fuses the decision with quantization format selection.

1.3 Prior Art

Chen et al. (2016) — the original gradient checkpointing paper. $\sqrt{N}$ memory in N -layer networks via strategic checkpoints. Established the save/recompute tradeoff but treated all activations uniformly.

Megatron selective recomputation (Korthikanti et al., 2022) — recognized that self-attention intermediates (softmax output, attention scores) are cheap to recompute but consume $O(\text{seq}^2)$ memory, while MLP activations are expensive to recompute but cheap to store. Achieves $\sim 5\times$ activation memory reduction at $\sim 5\%$ compute overhead. Still rule-based: user sets `recompute_granularity="selective"` and `recompute_modules=[...]` by hand.

Adacc (Chen et al., 2025) — first unified framework combining compression and recomputation. Uses MILP over tensor-level decisions, introduces outlier-separated compression for activations with heavy-tailed distributions, and adaptively revises the policy during training. Achieves $1.01\text{--}1.37\times$ speedup over SOTA Megatron. The MILP formulation is CCR's direct precedent.

PyTorch 2.4+ memory budget API (`torch._functorch.config.activation_memory_budget`) — exposes a user-facing memory fraction; the Inductor backend runs an automatic checkpointing solver. Reports note that it may not account for recompute memory overhead and can increase peak memory, but it is the most user-friendly production API.

TorchTitan (ICLR 2025) — composes selective activation checkpointing with FSDP, TP, PP, and torch.compile. Reference open-source implementation for large-scale training.

CCR occupies a different design point from all four: it combines Adacc's MILP unification with NSL's compile-time visibility of tensor roles, CFIP format awareness, and the composition contracts described in NSL's pipeline paper.

2. Seven Compiler-Native Recomputation Innovations

2.1 Innovation 1: Joint save/compress/recompute MILP

The central decision variable for tensor t is one of:

$$d_t \in \{\text{SAVE_FP}, \text{SAVE_CFIP}(f), \text{RECOMPUTE}\}$$

where f ranges over CFIP formats (IF4, IL2, IM3). The MILP objective minimizes training step time subject to the GPU memory budget:

$$\min \sum_t \text{step_time}(d_t) \quad \text{s.t.} \quad \sum_t \text{memory}(d_t) \leq B$$

Per-tensor step-time contribution depends on the decision:

- SAVE_FP: zero compute overhead, full memory cost.
- SAVE_CFIP(f): quantize-on-store cost + dequantize-on-load cost, reduced memory cost.
- RECOMPUTE: recompute cost $\times$ downstream use count, near-zero memory cost (only input deps).

The quantize/dequantize costs are hardware-specific; CCR's profile pass measures them per format on the target GPU during the first training step.

2.2 Innovation 2: Repetitive-block search-space reduction

A 32-layer transformer has 32 nearly-identical transformer blocks. Each block contains ~ 20 -30 named activation tensors. A naive MILP over all activations has $3 \cdot |F|$ options per tensor $\times \sim 800$ tensors = a huge search space.

CCR exploits the repetitive structure: solve the MILP for one representative block, apply the solution to all blocks. This reduces the MILP to ~ 25 variables with a handful of options each — solvable in milliseconds. Adacc's paper established this is a safe approximation; loss is within 0.5% of a full-graph solver across tested GPT configurations.

Non-repetitive parts of the model (embedding layer, output head, first/last blocks which often have slightly different behavior) get their own per-tensor MILP decisions.

2.3 Innovation 3: Outlier-aware compression format selection

Naive FP16-to-INT4 activation quantization can degrade GPT model accuracy by up to 39%, or cause gradient overflow in training (Adacc §4). The reason: activations exhibit systematic outliers along the channel dimension. A small number of channels (often $< 1\%$) carry values 10-100 $\times$ larger than the median.

CCR integrates with NSL's existing outlier-handling mechanisms:

- For tensors the compiler identifies as having heavy-tailed distributions (via the same analysis CFIP uses for GEBA), CCR prefers SAVE_CFIP(IM3) over SAVE_CFIP(IL2). IM3's wider dynamic range (3-bit mantissa, 2-bit exponent equivalent) handles outliers better than IL2's pure-linear encoding.
- For tensors with known channel-structured outliers (first-layer activations, post-LayerNorm tensors), CCR uses outlier-separated compression: the top-k outlier channels are stored uncompressed; the remaining channels go to the chosen CFIP format. This is a direct port of Adacc's outlier separation, integrated with CFIP's format-selection pass.
- For activations with rare-but-devastating outliers (attention-sink tokens), CCR defaults to RECOMPUTE over SAVE_CFIP to avoid the accuracy cliff.

The outlier analysis runs at compile time using a calibration trace from the first 100 training steps; the decisions are stable thereafter (outliers emerge early in pre-training and persist).

2.4 Innovation 4: Unified view with CSHA tiers

CSHA's three tiers are three points on the save/recompute spectrum:

- **Tier A** saves nothing, recomputes attention in backward (attention kernel is cheap and seq²-memory-heavy → ideal recompute candidate).
- **Tier B** saves the attention output; recomputes the softmax in backward.
- **Tier C** saves Q_proj, K_proj, V_proj, row_max, row_sum; backward reads them.

CCR's decision framework subsumes this: the attention layer's decision variable is "Tier A | Tier B | Tier C | Tier C with CFIP-in-HBM," chosen by the MILP rather than by user annotation. Existing `@csa(tier=...)` annotations become hints to the MILP (preferring one tier but allowing override if memory constraints require).

This unification eliminates the current situation where attention memory management and MLP memory management use completely different machinery.

2.5 Innovation 5: Composition with FASE memory release

When FASE is in Deferred mode, the gradient buffer is not allocated. The freed memory is published by M36's memory planner to CCR's MILP as additional budget. For a 7B model, this adds ~14 GB to CCR's memory budget, which typically moves decisions from RECOMPUTE toward SAVE_FP or SAVE_CFIP, reducing compute overhead.

The composition works in the opposite direction too: when CCR decides to SAVE many activations and consumes the memory budget, FASE may be forced to Immediate mode (because there is no memory headroom for FASE to use). CCR's MILP runs *before* FASE's mode selection; FASE reads CCR's memory commitment as a fixed input.

2.6 Innovation 6: Composition with CFIP for compressed saves

When CCR chooses `SAVE_CFIP(f)` for a tensor, the storage is a CFIP-format buffer in HBM — not a regular FP16 tensor. This is the "semi-interior CFIP" concept from the cross-paper connections paper: a CFIP format legally used outside a single kernel, under the constraint that the producer and consumer are both named and compile-time-known.

The save path emits a quantize-on-store kernel at the end of forward (fused with the tensor's producing kernel where possible); the load path emits a dequantize-on-load kernel at the start of the consuming backward kernel (fused with the backward kernel where possible). Both quantize and dequantize use M35 FP8 MMA machinery where the CFIP format is FP8-compatible; for IF4/IL2/IM3, dedicated element-wise PTX kernels are used.

2.7 Innovation 7: CTQS phase-triggered re-solve

CTQS phase transitions (training-step-count triggered, or loss-plateau triggered) can change the accuracy sensitivity of every tensor. An activation that was recomputable in phase 1 (because accuracy cost was acceptable at full precision) may need SAVE in phase 2 (when CTQS moves to FP8 everywhere and the cumulative compression budget is tighter).

CCR subscribes to CTQS's phase-change hook: when CTQS advances, CCR's MILP re-solves with the new per-tensor sensitivity weights. The re-solve is fast (milliseconds) because the MILP is small; new decisions are installed for the next training step.

This is the mechanism by which CCR's policy "evolves" during training, equivalent to Adacc's adaptive policy mechanism but with well-defined trigger semantics (CTQS phase events) rather than periodic re-profiling.

3. Implementation Architecture

3.1 Pipeline position

CCR runs in Stage 4 (WGGO), after fusion boundaries are finalized but before MIR lowering. Its output augments the WGGO decision table with per-tensor save/compress/recompute decisions.

Stage 4 (WGGO):

... existing per-layer decisions ...

→ CCR analysis pass:

1. Build activation memory graph from MIR preview
2. Profile per-tensor quantize/dequantize/recompute costs
3. Identify repetitive blocks
4. Solve per-block MILP
5. Apply solution to all repeated blocks
6. Solve per-tensor MILP for non-repetitive parts
7. Augment decision table

Stage 5 (MIR lowering):

... CCR decisions are read; MIR kernels gain save/load operations ...

3.2 The `@checkpoint` annotation

User-facing API is a single annotation at the model level:

```
ns!

@checkpoint(budget=0.5)      // 50% of default activation memory
model Transformer { ... }

@checkpoint(budget="auto")  // CCR chooses budget based on model size + available me
model Transformer { ... }

@checkpoint(budget=0.5, strategy="save_preferred") // bias toward saving
model Transformer { ... }
```

The `budget` parameter is the target activation memory as a fraction of default (no recomputation). `0.5` means "use half the memory I'd use without any recomputation, accepting whatever compute overhead this requires." `auto` lets CCR decide based on the total GPU memory budget minus everything else (weights, gradients, optimizer state).

3.3 MILP variables and constraints

For each activation tensor t in a representative block:

Variables:

- $x_{t,d} \in \{0, 1\}$: binary indicator that tensor t uses decision d .
- Constraint: $\sum_d x_{t,d} = 1$ (exactly one decision per tensor).

Where d ranges over:

- SAVE_FP16, SAVE_IF4, SAVE_IL2, SAVE_IM3, SAVE_FP8_E4M3, SAVE_FP8_E5M2, RECOMPUTE.

Objective:

$$\min \sum_t \sum_d x_{t,d} \cdot C(t, d)$$

where $C(t, d)$ is the step-time contribution of decision d for tensor t , measured during the profile pass.

Memory constraint:

$$\sum_t \sum_d x_{t,d} \cdot M(t, d) \leq B$$

where $M(t, d)$ is the memory cost and B is the budget.

Optional accuracy constraint (for strategy="accuracy_preferred"):

$$\sum_t \sum_d x_{t,d} \cdot A(t, d) \leq A_{\max}$$

where $A(t, d)$ is the accuracy-loss score for the decision and $A_{\max}$ is a user-set tolerance.

The MILP is solved using the `lp_solve` Rust crate or `good_lp` (integer programming). For a 25-variable MILP with 7 options each, solve time is <100 ms.

3.4 Runtime execution

The emitted backward kernels follow this pattern:

```
backward_kernel_i(output_grad, ...):
  for each activation a used by this backward:
    switch (ccr_decision[a]):
      case SAVE_FP16:
        load a from HBM (the saved buffer)
      case SAVE_CFIP(f):
        load a_quant from HBM
        dequantize a_quant to a (fused into this kernel's prologue)
      case RECOMPUTE:
        recompute a from its saved inputs (possibly recursively)

    ... use a to compute input gradients ...
```


Recomputation dependencies are resolved at compile time: if the MILP decides to RECOMPUTE tensor a from its inputs b, c , then b and c must have decisions that make them available (either SAVE_FP16, SAVE_CFIP, or themselves recomputable from a chain ending in inputs to the forward pass). The compile-time check ensures no cycles and no unresolvable chains.

3.5 Memory planner integration (M36)

M36's memory planner reads CCR's decisions and allocates accordingly:

- SAVE_FP16 tensors get HBM allocations at their declared size.
- SAVE_CFIP(f) tensors get HBM allocations at the format's compressed size.
- RECOMPUTE tensors get no allocation; only their input deps are allocated.

The planner schedules allocations to minimize peak memory: tensors with disjoint live ranges share the same HBM region. CCR's decisions feed into the existing liveness-analysis-based allocator without structural changes.

4. Integration with NSL Pipeline

4.1 Decision ordering relative to other techniques

CCR's place in the decision ordering (from the composition paper §5):

- After: datatype resolution (O-01), annotation resolution (O-02), CPFT freezing (O-03).
- Before: MIR lowering (O-04), fusion boundary finalization for CFIP (O-05), CTQS phase (O-06 — CCR re-runs on phase change).

This places CCR in the middle of WGGO, after WGGO has made per-layer decisions (which feed CCR's tensor profile) but before MIR kernels are emitted.

4.2 New invariants

Adding to the composition paper's numbered list:

I-24 — CCR decisions are globally consistent. A tensor's CCR decision must be consistent with the decisions of its recompute dependencies. RECOMPUTE of a from $\{b, c\}$ requires b and c to be available (SAVE_* or recursively RECOMPUTE with terminating chain).

I-25 — CCR memory commitment is read-only after Stage 4. Once CCR finalizes, M36 and all downstream techniques treat the memory commitment as fixed.

I-26 — CTQS phase changes trigger CCR re-solve. CCR subscribes to CTQS's phase-change event; the new MILP solution is installed before the next training step begins.

I-27 — CFIP format compatibility is checked before SAVE_CFIP decisions. CCR may only propose SAVE_CFIP(f) for tensors whose GEBA bound admits format f . CFIP's GEBA analysis runs as a prerequisite.

4.3 Failure modes

F-15 — Recompute chain cycle. Symptom: compile-time error about circular recompute dependency. Root cause: MILP decided RECOMPUTE for a and b , where a is a recompute input for b and vice versa. Mitigation: the MILP includes a DAG constraint ensuring no cycles. Caught at solver time.

F-16 — SAVE_CFIP accuracy degradation at phase change. Symptom: loss jumps at a CTQS phase transition. Root cause: a tensor's accuracy sensitivity changed but CCR's re-solve was stale. Mitigation: CCR's re-solve is synchronous with the phase change (§2.7); no stale decisions persist.

F-17 — Memory budget not achievable. Symptom: compile-time error reporting MILP infeasibility. Root cause: the budget is too tight for the model (every activation in RECOMPUTE still exceeds the budget). Mitigation: compile-time error with actionable message suggesting a larger budget or smaller batch.

5. Evaluation Plan

5.1 Baseline comparisons

Comparisons against:

1. **Baseline (no recomputation):** peak memory baseline for speed; fails OOM at larger batches.
2. **Megatron full recomputation:** coarse; $\sim 30\%$ compute overhead at $\sim 5\times$ memory reduction.
3. **Megatron selective recomputation:** $\sim 5\%$ compute overhead at $\sim 5\times$ memory reduction.
4. **PyTorch 2.4 memory budget API at 0.5:** automated; variable overhead.
5. **Adacc MILP:** closest academic peer; $1.01\text{--}1.37\times$ speedup over Megatron selective.
6. **CCR (this work):** expected to match or exceed Adacc on end-to-end training throughput at equivalent memory.

5.2 Benchmark matrix

Model	Params	Seq	Batch	Target GPU
NSLCoder-50M	50M	2048	16	RTX 5070 Ti 16GB
NSL-400M test	400M	2048	8	RTX 5070 Ti 16GB
Llama-style 7B	7B	2048	4	H100 80GB
Llama-style 7B	7B	8192	2	H100 80GB (seq-scaling test)

5.3 Metrics

- **Peak activation memory (GB):** MILP's claimed memory vs observed peak.
- **Compute overhead (%):** training step time vs baseline (no recomputation).
- **Training accuracy:** final loss / perplexity after N steps, compared to baseline.
- **MILP solve time (ms):** should be <100ms per solve; re-solves on phase change.

5.4 Correctness tests

1. **Bit-exact save/recompute equivalence at FP16:** MILP decisions with all SAVE_FP16 must produce bitwise-identical results to baseline.
2. **Numerical equivalence at FP8:** MILP with SAVE_CFIP(FP8_E4M3) decisions produces results within declared tolerance of FP16 baseline.
3. **Outlier preservation:** for tensors flagged as outlier-heavy, SAVE_CFIP with outlier separation preserves the outlier values to within 0.01% relative error.
4. **Composition with FASE Deferred:** CCR + FASE Deferred produces identical final weights to CCR + FASE Immediate at matched budget.
5. **CTQS phase transition smoothness:** loss curve across a phase transition shows no discontinuity larger than 0.5% (above which, CTQS's loss guard triggers a rollback).

5.5 Implementation phases

Phase	Scope	Sessions
1	CCR analysis pass infrastructure, MILP setup	2
2	Profile pass (measure per-tensor costs)	1
3	MILP solver integration (lp_solve / good_lp)	1
4	M36 memory planner integration	1
5	Runtime save/load kernels (FP16 path)	1
6	CFIP format integration (SAVE_CFIP)	2
7	Outlier-separated compression	1
8	CSHA tier unification	1
9	FASE / CPFT / CTQS composition	1
10	Correctness and benchmark tests	2
Total		13

Estimated total scope: ~2800 LOC across 6 files, 13 implementation sessions.

6. Limitations and Future Work

6.1 Known limitations

MILP scalability at extreme model scales. For very large models with hundreds of non-repetitive components (MoE with many distinct experts, heterogeneous layer stacks), the MILP may grow beyond the 1-second solve budget. Fallback: a greedy heuristic with Adacc-style block-level aggregation.

Profile-pass cost at first run. The per-tensor cost profile requires one full training step at baseline, which takes seconds to minutes depending on model size. For rapid iteration, a cached profile from a previous compile can be reused if the model architecture is unchanged.

Dynamic sequence length. Current formulation assumes sequence length is fixed or within a declared range. Fully dynamic sequences (each request a different length) require runtime re-solve, which is not yet supported.

6.2 Future work

Learned cost model instead of profiling: a small ML model predicts per-tensor costs from tensor shape and kernel type. Eliminates the profile pass. Training data: cached profiles from previous NSL compiles.

Pipeline-parallel activation management: in pipeline-parallel training, activations must be held until the backward wave arrives. CCR's MILP currently treats this as additional live-range burden; a future extension explicitly models pipeline-parallel interleaving.

Cross-step recomputation: activations from step $N - 1$ reused in step N 's forward (rare but appears in some curriculum-learning setups). CCR currently treats steps independently.

7. Conclusion

Activation memory is the single largest consumer of training GPU memory after optimizer state and the largest *unaddressed* consumer in NSL. CCR fills this gap with a MILP-based decision framework that unifies save, compress, and recompute choices into one compile-time optimization, leveraging NSL's existing infrastructure (CFIP formats, CSHA tiers, FASE memory release, CPFT freezing, CTQS phase transitions, M36 memory planning). The expected 2-5 $\times$ memory reduction at 2-10% compute overhead matches or exceeds external baselines while integrating cleanly with every other NSL technique. The MILP's repetitive-block search-space reduction keeps solve time in the sub-100ms range, making CCR compatible with fast iteration workflows. With CCR shipped, NSL addresses every major consumer of training GPU memory: weights (CFTP/CFIP), gradients (FASE), optimizer state (CPDT ZeRO), and activations (CCR).

References

1. Chen, T. et al. (2016). *Training Deep Nets with Sublinear Memory Cost*. arXiv:1604.06174.
2. Korthikanti, V. et al. (2022). *Reducing Activation Recomputation in Large Transformer Models*. arXiv:2205.05198.
3. Chen, P. et al. (2025). *Adacc: An Adaptive Framework Unifying Compression and Activation Recomputation for LLM Training*. arXiv:2508.00806.
4. PyTorch 2.4 Documentation. `torch._functorch.config.activation_memory_budget`.
5. Liang, C. et al. (2025). *TorchTitan: One-stop PyTorch Native Solution for Production Ready LLM Pre-training*. ICLR 2025.
6. Jain, P. et al. (2019). *Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization*. MLSys 2020.